

EVALUASI AKHIR SEMESTER - GENAP 2025 / 2026
DEPARTEMEN MATEMATIKA - FSAD ITS



Mata kuliah / Kode : ALGORITMA DAN PEMROGRAMAN 2
 Hari / Tanggal : KAMIS / 18 JUNI 2026
 Sifat / waktu : TUTUP BUKU / 100 menit
 Dosen : Dr. Dwi Ratna Sultyaningrum, S.Si., M.T.
 Dr. Budl Setiyono, S.Si., M.T.
 Alvilda Mustika Rukmil, S.Si., M.Si.
 Mohammad Iqbal, S.Si., M.Si., Ph.D.



HARAP DIPERHATIKAN !!!

Segala jenis pelanggaran (mencontek, kerjasama, dsb) yang dilakukan saat ETS/EAS akan dikenakan sanksi pembatalan mata kuliah pada semester yang sedang berjalan.

EAS mengukur kemampuan

CPMK		BOBOT CPMK (%)	CPL				
			4	5	7	8	9
1	Mampu memahami dasar-dasar untuk membangun sebuah algoritma yang benar dan efisien						
2	Mampu memahami dan menganalisa algoritma <i>searching</i> dan <i>sorting</i>						
3	Mampu memahami dan mengimplementasikan <i>Exception Handling</i>						
4	Mampu memahami dan mengimplementasikan GUI						
5	Mampu memahami konsep dasar pemrograman berorientasi objek	25				12.5	12.5
6	Mampu menerapkan paradigma pemrograman berorientasi objek dalam mengembangkan suatu program untuk menyelesaikan suatu masalah secara individu maupun tim	20			10		10

SOAL	BOBOT SOAL (%)	CPMK-1	CPMK-2	CPMK-3	CPMK-4
1	20			5	
2	20			5	5
3	20			5	5
4	20			5	5
5	20			5	5
TOTAL BOBOT CPMK				25	20

1. Enkapsulasi

Perhatikan kode yang diberikan:

```
class Mahasiswa {
    public String nama;
    public double ipk;
}
```

Kemudian diubah menjadi:

```
class Mahasiswa {
    private String nama;
    private double ipk;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }
}
```

- a) Apa tujuan perubahan tersebut?
- b) Apa keuntungan penggunaan *modifier private*?
- c) Jelaskan manfaat dari teknik enkapsulasi diatas.

2. Pewarisan (Inheritance)

```
class Barang {
    private String nama;
    private double harga;

    Barang(String nama, double harga) {
        // lengkapi;
    }

    public double getHarga() {
        return harga;
    }
}

class Toko {
    public static void main(String[] args)
    ↪ {
        HargaDiskon produk = new
        ↪ HargaDiskon("Detergen", 15000,
        ↪ 0.2);
        // lengkapi;
    }
}
```

```
class HargaDiskon extends Barang {
    private double diskon;

    HargaDiskon(String nama, double harga,
    ↪ double diskon) {
        // lengkapi;
    }

    public double nettoHarga(int
    ↪ jumlahBarang) {
        // lengkapi;
    }
}
```

Lengkapi program di atas pada:

- a) Constructor pada kelas Barang dan kelas HargaDiskon
- b) Method `nettoHarga` dimana jumlah barang ≥ 8 maka mendapatkan diskon.
- c) Kelas Toko untuk menampilkan output: Harga Netto = 12000 dengan memasukkan jumlah barang 10.

3. Polimorfisme (Polymorphism)

Perhatikan listing program dibawah, jawab pertanyaan berikut :

```
package EAS2026;
public class SoalPolymorphisme {
    public static void main(String[] args) {
        Kendaraan s = new Sepeda();      // baris A
        s.info();
        s.jalan();
        Kendaraan k = new Mobil();       // baris B
        k.jalan();
        //k.klakson();                   // baris C (di-comment)
    }
}

class Kendaraan {
    void info() { System.out.println("Kendaraan umum"); }
    void jalan() { System.out.println("Tancap gas ...."); }
}

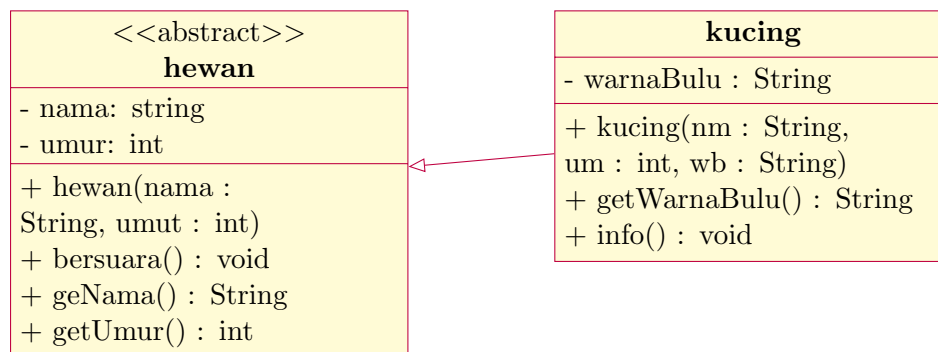
class Mobil extends Kendaraan {
    @Override
    void info() { System.out.println("Ini Mobil"); }
    void klakson() { System.out.println("Tin tin!"); }
}

class Sepeda extends Kendaraan {
    @Override
    void info() { System.out.println("Ini Sepeda"); }
    void jalan() { System.out.println("Putar gas ...."); }
}
```

- Apa yang terjadi pada baris A dan baris B? Sebutkan istilah teknisnya!
- Mengapa baris C akan error jika di-*uncomment*? Jelaskan!
- Tuliskan output program diatas.

4. Abstract dan Interface

Perhatikan class diagram berikut :



- a) Implementasikan class diagram tersebut

Keterangan :

- **bersuara** akan mencetak “meong”
- **info** akan mencetak nama, umur, warna bulu dan bersuara

- b) Buat sebuah obyek, lalu dengan menggunakan obyek tersebut jalankan method **info**.

5. Enkapsulasi

Desain Kelas untuk sistem perpustakaan. Perhatikan skenario yang diberikan.

Skenario

Perpustakaan kampus ingin membuat sistem peminjaman buku. Setiap buku memiliki kode ISBN (unik, tidak berubah), judul, nama pengarang, dan status ketersediaan (tersedia atau dipinjam). Status hanya boleh berubah melalui dua aksi resmi: pinjam dan kembalikan. Ketika buku dipinjam, sistem harus memastikan buku dalam kondisi tersedia. Ketika dikembalikan, buku harus dalam kondisi sedang dipinjam.

- a) Rancang daftar atribut kelas Buku beserta akses modifier yang tepat untuk masing-masing. Jelaskan pilihan Anda!
- b) Mengapa atribut status ketersediaan tidak boleh memiliki setter public biasa seperti `setStatus(Boolean status)`? Apa risikonya jika dibuat?
- c) Jika kelas lain ingin mengecek apakah buku tersedia sebelum meminjam, mekanisme apa yang sebaiknya disediakan kelas Buku?

SOLUSI

1. a) Tujuan perubahan adalah membatasi akses langsung ke data objek dan memaksa akses melalui method yang terkontrol. Dengan begitu data lebih aman dan aturan validasi bisa diterapkan.
b) Keuntungan **private** adalah atribut tidak bisa diubah sembarangan dari luar kelas, sehingga integritas data terjaga dan implementasi internal bisa diubah tanpa memengaruhi pemakai kelas.
c) Manfaat enkapsulasi adalah menyembunyikan detail implementasi, menjaga konsistensi objek, memudahkan perawatan kode, dan membuat kelas lebih aman untuk digunakan.

2. a) Constructor pada kelas **Barang** dan kelas **HargaDiskon**:

```
Barang(String nama, double harga) {  
    this.nama = nama;  
    this.harga = harga;  
}  
  
HargaDiskon(String nama, double harga, double diskon) {  
    super(nama, harga);  
    this.diskon = diskon;  
}
```

- b) Method **nettoHarga**:

```
public double nettoHarga(int jumlahBarang) {  
    if (jumlahBarang >= 8) {  
        return getHarga() - (getHarga() * diskon);  
    }  
    return getHarga();  
}
```

- c) Kelas **Toko**:

```
class Toko {  
    public static void main(String[] args) {  
        HargaDiskon produk = new HargaDiskon("Detergen", 15000, 0.2);  
        System.out.println("Harga Netto = " + produk.nettoHarga(10));  
    }  
}
```

3. a) Baris A dan B melakukan *upcasting*, yaitu objek turunan disimpan ke referensi kelas induk. Saat method di-override, pemanggilan method mengikuti objek sebenarnya pada runtime, sehingga terjadi *dynamic dispatch*.
b) Baris C error jika di-uncomment karena variabel **k** bertipe **Kendaraan**, sedangkan **Kendaraan** tidak memiliki method **klakson()**. Method tersebut hanya ada di **Mobil**, jadi tidak bisa dipanggil lewat referensi induk.
c) Output program:

```
Ini Sepeda  
Putar gas ....  
Tancap gas ....
```

4. a) Implementasi class Hewan dan Kucing:

```
abstract class Hewan {
    private String nama;
    private int umur;

    Hewan(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    public String getNama() {
        return nama;
    }

    public int getUmur() {
        return umur;
    }

    public abstract void bersuara();
}

class Kucing extends Hewan {
    private String warnaBulu;

    Kucing(String nama, int umur, String warnaBulu) {
        super(nama, umur);
        this.warnaBulu = warnaBulu;
    }

    public String getWarnaBulu() {
        return warnaBulu;
    }

    @Override
    public void bersuara() {
        System.out.println("meong");
    }

    public void info() {
        System.out.println("Nama: " + getNama());
        System.out.println("Umur: " + getUmur());
        System.out.println("Warna bulu: " + getWarnaBulu());
        bersuara();
    }
}
```

b) Membuat objek dan memanggil method info:

```
class DemoKucing {
    public static void main(String[] args) {
        Kucing kucing = new Kucing("Mimi", 2, "Putih");
        kucing.info();
    }
}
```

```
}
```

5. a) Atribut kelas Buku yang dirancang:

- **private final String isbn:** *Modifier private* untuk menyembunyikan data dari luar kelas. Tambahan **final** digunakan karena kode ISBN bersifat unik untuk setiap buku dan tidak boleh berubah setelah objek buku dibuat.
 - **private String judul:** *Modifier private* untuk menjaga enkapsulasi, sehingga judul hanya bisa diakses/diubah (jika diperlukan) melalui *method* yang disediakan.
 - **private String pengarang:** Sama seperti judul, **private** digunakan untuk melindungi data agar tidak diakses langsung dari luar.
 - **private boolean tersedia:** *Modifier private* sangat krusial di sini agar status peminjaman tidak bisa diubah langsung dari luar. Perubahan status hanya diizinkan melalui *method* resmi yaitu **pinjam()** dan **kembalikan()**.
- b) **status** tidak boleh punya *setter public* biasa (seperti **setStatus(boolean status)**) karena hal tersebut akan merusak logika bisnis peminjaman. Risiko jika dibuat: kelas lain bisa sembarangan mengubah status buku yang sedang dipinjam menjadi tersedia (atau sebaliknya) tanpa melalui proses validasi (misal mengecek apakah buku benar-benar sedang dipinjam sebelum dikembalikan).
- c) Mekanisme yang sebaiknya disediakan adalah *method getter/accessor* yang mengembalikan nilai boolean, contohnya **isTersedia()** atau **getTersedia()**. Dengan begitu, kelas lain bisa mengecek kondisi buku tanpa memiliki akses untuk mengubah nilainya.